

p1_report

February 20, 2026

Author: Yuze Fu

The goal of this project is to test 8 finite difference schemes and get their order of convergence and stability conditions.

We experiment schemes on an advection function, typically with initial condition and periodic boundary condition as follows:

$$\begin{cases} u_t(x, t) + cu_x(x, t) = 0 \\ u(x, 0) = g(x) \\ u(a, t) = u(b, t) \end{cases}$$

In this project, we fix $g(x) = e^{ix}$ which is the complex unit circle.

1 Code

These are the code for the implementations of the 8 finite difference schemes in *np.roll()* approach, which is faster than matrix or loop approach.

```
[10]: import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import solve_circulant

def g(x):
    return np.exp(1j * x)

def run_upwind(nx, CFL, T, c=1.0, a=-np.pi, b=np.pi):
    dx = (b - a) / nx
    xs = np.linspace(a, b, nx, endpoint=False)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u = g(xs)
    for _ in range(nt):
        if c > 0:
            u = u - nu * (u - np.roll(u, 1))
        else:
            u = u - nu * (np.roll(u, -1) - u)
```

```

u_exact = g(a + np.mod(xs - c * T - a, b - a))
error = np.max(np.abs(u - u_exact))
return xs, u, u_exact, dx, error

def run_cn(nx, CFL, T, c=1.0, a=-np.pi, b=np.pi):
    dx = (b - a) / nx
    xs = np.linspace(a, b, nx, endpoint=False)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt

    u = np.zeros(nx, dtype="complex")
    v = np.zeros(nx, dtype="complex")
    nu = (c / 4) * (dt / dx)

    s = np.zeros(nx)
    s[-1] = nu
    s[0] = 1.0
    s[1] = -nu

    u = g(xs)

    for j in range(nt):
        v = u - nu * (np.roll(u, -1) - np.roll(u, 1))
        u = solve_circulant(s, v)

    u_exact = g(a + np.mod(xs - c * T - a, b - a))
    error = np.max(np.abs(u - u_exact))
    return xs, u, u_exact, dx, error

def run_lw(nx, CFL, T, c=1.0, a=-np.pi, b=np.pi):
    dx = (b - a) / nx
    xs = np.linspace(a, b, nx, endpoint=False)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u = np.zeros(nx, dtype="complex")
    u = g(xs)

    for j in range(nt):
        u = u - (nu / 2) * (np.roll(u, -1) - np.roll(u, 1)) + (nu**2 / 2) * (np.
↪roll(u, -1) - 2 * u + np.roll(u, 1))

    u_exact = g(a + np.mod(xs - c * T - a, b - a))

```

```

    error = np.max(np.abs(u - u_exact))
    return xs, u, u_exact, dx, error

def run_lf(nx, CFL, T, c=1.0, a=-np.pi, b=np.pi):
    dx = (b - a) / nx
    xs = np.linspace(a, b, nx, endpoint=False)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u1 = np.zeros(nx, dtype="complex")
    u1 = g(xs)
    u2 = u1 - nu * (u1 - np.roll(u1, 1))

    for j in range(0, nt - 1):
        temp = u2
        u2 = -nu * (np.roll(u2, -1) - np.roll(u2, 1)) + u1
        u1 = temp

    u_exact = g(a + np.mod(xs - c * T - a, b - a))
    error = np.max(np.abs(u2 - u_exact))
    return xs, u2, u_exact, dx, error

def run_ftcds(nx, CFL, T, c=1.0, a=-np.pi, b=np.pi):
    dx = (b - a) / nx
    xs = np.linspace(a, b, nx, endpoint=False)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u = g(xs)
    for j in range(0, nt - 1):
        u = u - (nu / 2) * (np.roll(u, -1) - np.roll(u, 1))

    u_exact = g(a + np.mod(xs - c * T - a, b - a))
    error = np.max(np.abs(u - u_exact))
    return xs, u, u_exact, dx, error

def run_laxf(nx, CFL, T, c=1.0, a=-np.pi, b=np.pi):
    dx = (b - a) / nx
    xs = np.linspace(a, b, nx, endpoint=False)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

```

```

u = g(xs)
for j in range(nt):
    u = (1 / 2) * (np.roll(u, -1) + np.roll(u, 1)) - (nu / 2) * (np.roll(u, 1)
    ↪ - 1) - np.roll(u, 1))

u_exact = g(a + np.mod(xs - c * T - a, b - a))
error = np.max(np.abs(u - u_exact))
return xs, u, u_exact, dx, error

def run_rk2(nx, CFL, T, c=1.0, a=-np.pi, b=np.pi):
    dx = (b - a) / nx
    xs = np.linspace(a, b, nx, endpoint=False)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u = g(xs)
    for j in range(0, nt):
        d = np.roll(u, -1) - np.roll(u, 1)
        u = u - (nu / 2) * d + (nu ** 2 / 8) * (np.roll(d, -1) - np.roll(d, 1))

    u_exact = g(a + np.mod(xs - c * T - a, b - a))
    error = np.max(np.abs(u - u_exact))
    return xs, u, u_exact, dx, error

def L(u, dx):
    return (-np.roll(u, -2) + 8*np.roll(u, -1) - 8*np.roll(u, 1) + np.roll(u, 2)
    ↪ ) / (12 * dx)

def run_rk4(nx, CFL, T, c=1.0, a=-np.pi, b=np.pi):
    dx = (b - a) / nx
    xs = np.linspace(a, b, nx, endpoint=False)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u = np.zeros(nx, dtype="complex")
    u = g(xs)

    for _ in range(nt):
        k1 = -c * L(u, dx)
        k2 = -c * L(u + 0.5 * dt * k1, dx)
        k3 = -c * L(u + 0.5 * dt * k2, dx)
        k4 = -c * L(u + dt * k3, dx)

```

```

    u = u + (dt / 6) * (k1 + 2*k2 + 2*k3 + k4)

u_exact = g(a + np.mod(xs - c * T - a, b - a))
error = np.max(np.abs(u - u_exact))
return xs, u, u_exact, dx, error

```

2 Experiments

The following is my design for the three tasks:

2.1 Order of Convergence (Space)

Based on the definition, we have $E \approx C(\Delta x)^p$, and after applying log, we get $\log(E) \approx \log(C) + p \log(\Delta x)$. Thus, my experiment is when fixing the other variables, double nx each time and get its corresponding dx and $error$. Then, rescale them by log and check the slope for $error/dx$. If the slope is p , then it's p -th order of convergence. How I implement this is plot the log value and compare it with $y = px$.

2.2 Stability properties

Change one of CFL, T, c, nx and fixing the other variables. Print both the exact value and numerical value from a to b at nt and compare (e.g. oscillations, shifting). Though I only show the graphs for CFL changing, I also test on other variables by myself.

2.3 CFL vs. Stability

By our choice of initial condition $u(x, 0) = g(x) = e^{ix}$ whose norm is 1, our numerical value should be approximately equal to 1 as well. The design of this experiment is to fix other variables and only focus on how CFL affects the norm of numerical value. Ranging a series of CFL values from low to high (e.g. 0.1 -> 1.2), get the maximum norm of the numerical value at nt . Plot them and check when the norm blows up, and when the max norm is approximately equal to 1, it means the CFL number can keep the scheme stable.

3 Graph and Discussion

3.1 Upwind

```

[11]: # Experiment 1: Convergence (Error vs. dx)
nxs = [50, 100, 200, 400, 800]
dxs, errors = [], []
for n in nxs:
    _, _, _, dx, err = run_upwind(n, CFL=0.8, T=15.0)
    dxs.append(dx)
    errors.append(err)

# Experiment 2 & 3: Stability and CFL Effects
cfl_test_vals = np.linspace(0.1, 1.2, 20)
max_amplitudes = []

```

```

for cfl in cfl_test_vals:
    _, u_final, _, _, _ = run_upwind(nx=100, CFL=cfl, T=15.0)
    max_amplitudes.append(np.max(np.abs(u_final)))

# Visualization
fig, axs = plt.subplots(1, 4, figsize=(18, 5))

# Plot 1: Convergence
axs[0].loglog(dxs, errors, 'o-', label='Upwind Error')
axs[0].loglog(dxs, [d for d in dxs], '--', label='Order 1 (Slope=1)')
axs[0].set_title("Exp 1: Order of Convergence (CFL=0.8, T=15.0)")
axs[0].set_xlabel("dx")
axs[0].set_ylabel("Max Error")
axs[0].legend()

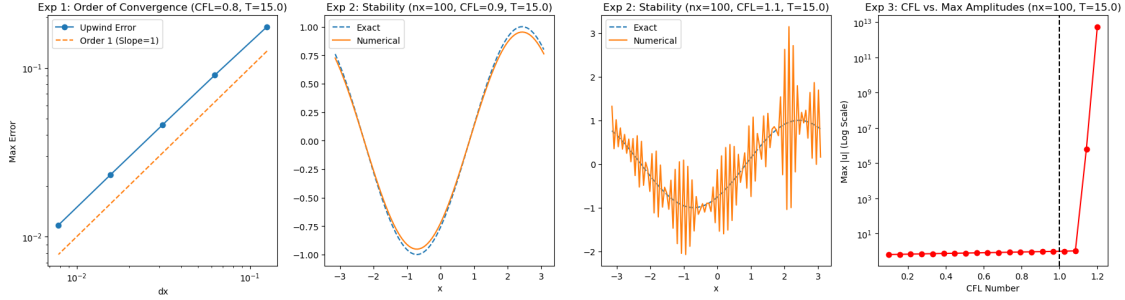
# Plot 2: Stability Identification
xs, u, u_ex, _, _ = run_upwind(nx=100, CFL=0.9, T=15.0)
axs[1].plot(xs, u_ex.real, '--', label='Exact')
axs[1].plot(xs, u.real, label='Numerical')
axs[1].set_title("Exp 2: Stability (nx=100, CFL=0.9, T=15.0)")
axs[1].set_xlabel("x")
axs[1].legend()

xs, u, u_ex, _, _ = run_upwind(nx=100, CFL=1.1, T=15.0)
axs[2].plot(xs, u_ex.real, '--', label='Exact')
axs[2].plot(xs, u.real, label='Numerical')
axs[2].set_title("Exp 2: Stability (nx=100, CFL=1.1, T=15.0)")
axs[2].set_xlabel("x")
axs[2].legend()

# Plot 3: CFL vs. Stability
axs[3].plot(cfl_test_vals, max_amplitudes, 'ro-')
axs[3].axvline(1.0, color='k', linestyle='--')
axs[3].set_yscale('log')
axs[3].set_title("Exp 3: CFL vs. Max Amplitudes (nx=100, T=15.0)")
axs[3].set_xlabel("CFL Number")
axs[3].set_ylabel("Max |u| (Log Scale)")

plt.tight_layout()
plt.show()

```



- As we can see in the graph, the upwind scheme is order 1 convergence.
- For CFL , when we increase it from 0 to 1, the numerical wave approaches the exact wave. However, after some limit bigger than 1, the wave becomes oscillations. For T and c , when we increase them, the numerical wave will become flatter. For nx , when we increase it, the numerical wave will approach the exact wave.
- From last graph, we can see that the norm blows up at some point around $CFL = 1.1$. After that, the scheme becomes significantly unstable.
- In general, after dealing with c by cases, the upwind scheme performs fair if the variables are proper (ie. $CFL \approx 1^-$, $T < 50$, $c < 0$, $nx > 100$) which serves as a baseline for solving PDE with periodic condition.

3.2 Crank-Nicholson

```
[12]: # Experiment 1: Convergence (Error vs. dx)
nxs = [50, 100, 200, 400, 800]
dxs, errors = [], []
for n in nxs:
    _, _, _, dx, err = run_cn(n, CFL=0.5, T=2.0)
    dxs.append(dx)
    errors.append(err)

# Experiment 2 & 3: Stability and CFL Effects
cfl_test_vals = np.linspace(0.1, 5.0, 50)
max_amplitudes = []
for cfl in cfl_test_vals:
    _, u_final, _, _, _ = run_cn(nx=100, CFL=cfl, T=15.0)
    max_amplitudes.append(np.max(np.abs(u_final)))

# Visualization
fig, axs = plt.subplots(1, 4, figsize=(18, 5))

# Plot 1: Convergence
axs[0].loglog(dxs, errors, 'o-', label='CN Error')
```

```

axs[0].loglog(dxs, [d**2 for d in dxs], '--', label='Order 2 (Slope=2)')
axs[0].set_title("Exp 1: Order of Convergence")
axs[0].set_xlabel("dx")
axs[0].set_ylabel("Max Error")
axs[0].legend()

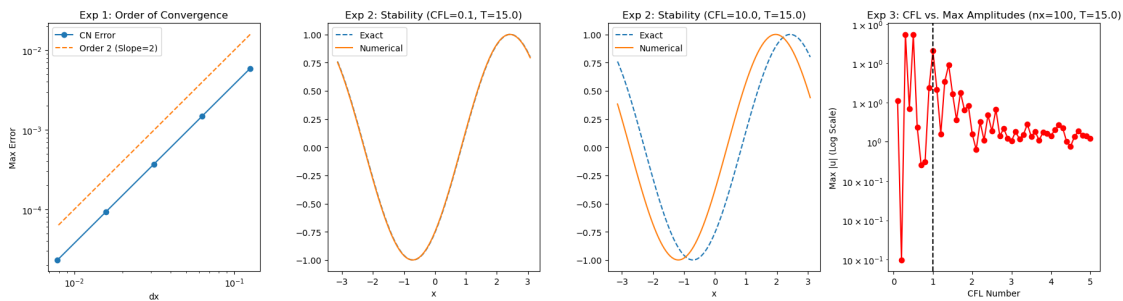
# Plot 2: Stability (Visualizing the Complex Wave)
xs, u, u_ex, _, _ = run_cn(nx=100, CFL=0.1, T=15.0)
axs[1].plot(xs, u_ex.real, '--', label='Exact')
axs[1].plot(xs, u.real, label='Numerical')
axs[1].set_title("Exp 2: Stability (CFL=0.1, T=15.0)")
axs[1].set_xlabel("x")
axs[1].legend()

xs, u, u_ex, _, _ = run_cn(nx=100, CFL=10.0, T=15.0)
axs[2].plot(xs, u_ex.real, '--', label='Exact')
axs[2].plot(xs, u.real, label='Numerical')
axs[2].set_title("Exp 2: Stability (CFL=10.0, T=15.0)")
axs[2].set_xlabel("x")
axs[2].legend()

# Plot 3: CFL vs. Stability
axs[3].plot(cfl_test_vals, max_amplitudes, 'ro-')
axs[3].axvline(1.0, color='k', linestyle='--')
axs[3].set_yscale('log')
axs[3].set_title("Exp 3: CFL vs. Max Amplitudes (nx=100, T=15.0)")
axs[3].set_xlabel("CFL Number")
axs[3].set_ylabel("Max |u| (Log Scale)")

plt.tight_layout()
plt.show()

```



- As we can see in the graph, the Crank-Nicholson scheme is order 2 convergence.
- For nx , when we increase it, the numerical wave will approach the exact wave. But for CFL, T and c , increasing them will not affect the amplitudes. It will only increase the delay which represent as left-right shifting on the graph.

- From last graph, we can see that the norm gradually converge to 1 as CFL number increases, which can be explained by the derivation of this scheme.
- In general, Crank Nicholson scheme converges faster than order 1 scheme such as upwind, and won't cause the "blow up" problem. However, the delay created by large variables is still a concern, and we should keep CFL, T, c in a range.

3.3 Lax-Wendroff

```
[13]: # Experiment 1: Convergence (Error vs. dx)
nxs = [50, 100, 200, 400, 800]
dxs, errors = [], []
for n in nxs:
    _, _, _, dx, err = run_lw(n, CFL=0.8, T=15.0)
    dxs.append(dx)
    errors.append(err)

# Experiment 2 & 3: Stability and CFL Effects
cfl_test_vals = np.linspace(0.1, 1.2, 20)
max_amplitudes = []
for cfl in cfl_test_vals:
    _, u_final, _, _, _ = run_lw(nx=100, CFL=cfl, T=15.0)
    max_amplitudes.append(np.max(np.abs(u_final)))

# Visualization
fig, axs = plt.subplots(1, 4, figsize=(18, 5))

# Plot 1: Convergence
axs[0].loglog(dxs, errors, 'o-', label='LW Error')
axs[0].loglog(dxs, [d**2 for d in dxs], '--', label='Order 2 (Slope=2)')
axs[0].set_title("Exp 1: Order of Convergence (CFL=0.8, T=15.0)")
axs[0].set_xlabel("dx")
axs[0].set_ylabel("Max Error")
axs[0].legend()

# Plot 2: Stability Identification
xs, u, u_ex, _, _ = run_lw(nx=100, CFL=0.95, T=15.0)
axs[1].plot(xs, u_ex.real, '--', label='Exact')
axs[1].plot(xs, u.real, label='Numerical')
axs[1].set_title("Exp 2: Stability (nx=100, CFL=0.95, T=15.0)")
axs[1].set_xlabel("x")
axs[1].legend()

xs, u, u_ex, _, _ = run_lw(nx=100, CFL=1.05, T=15.0)
axs[2].plot(xs, u_ex.real, '--', label='Exact')
axs[2].plot(xs, u.real, label='Numerical')
axs[2].set_title("Exp 2: Stability (nx=100, CFL=1.05, T=15.0)")
```

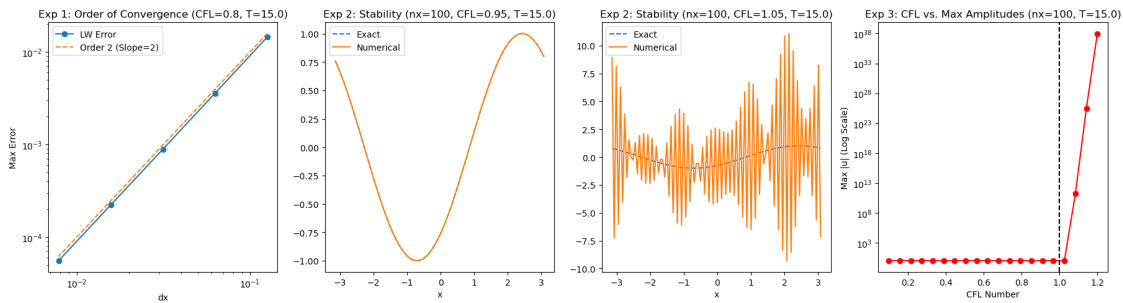
```

axs[2].set_xlabel("x")
axs[2].legend()

# Plot 3: CFL vs. Stability
axs[3].plot(cfl_test_vals, max_amplitudes, 'ro-')
axs[3].axvline(1.0, color='k', linestyle='--')
axs[3].set_yscale('log')
axs[3].set_title("Exp 3: CFL vs. Max Amplitudes (nx=100, T=15.0)")
axs[3].set_xlabel("CFL Number")
axs[3].set_ylabel("Max |u| (Log Scale)")

plt.tight_layout()
plt.show()

```



- As we can see in the graph, the Lax-Wendroff scheme is order 2 convergence.
- For CFL , when we increase it from 0 to 1, the numerical wave approaches the exact wave. However, after some limit bigger than 1, the wave becomes oscillations. For nx , when we increase it, the numerical wave will approach the exact wave. But for CFL, c , increasing them will not affect the amplitudes. It will only increase the delay which represent as left-right shifting on the graph.
- From last graph, we can see that the norm blows up at some point around $CFL = 1.05$. After that, the scheme becomes significantly unstable.
- In general, the convergence order and stability conditions of Lax-Wendroff scheme is similar to Crank-Nicholson scheme. However, controlling CFL is necessary for Lax-Wendroff, which may cause instability if it's larger than 1.

3.4 Leapfrog

```

[14]: # Experiment 1: Convergence (Error vs. dx)
nxs = [50, 100, 200, 400, 800]
dxs, errors = [], []
for n in nxs:
    _, _, _, dx, err = run_lf(n, CFL=0.8, T=15.0)

```

```

dxs.append(dx)
errors.append(err)

# Experiment 2 & 3: Stability and CFL Effects
cfl_test_vals = np.linspace(0.1, 1.05, 40)
max_amplitudes = []
for cfl in cfl_test_vals:
    _, u_final, _, _, _ = run_lf(nx=100, CFL=cfl, T=15.0)
    max_amplitudes.append(np.max(np.abs(u_final)))

# Visualization
fig, axs = plt.subplots(1, 4, figsize=(18, 5))

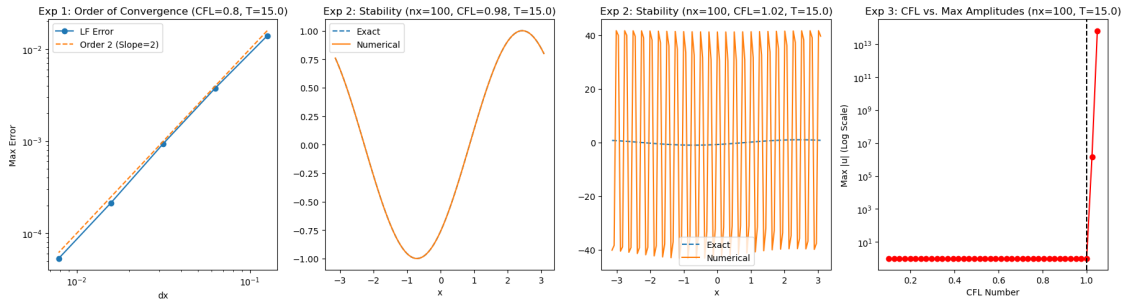
# Plot 1: Convergence
axs[0].loglog(dxs, errors, 'o-', label='LF Error')
axs[0].loglog(dxs, [d**2 for d in dxs], '--', label='Order 2 (Slope=2)')
axs[0].set_title("Exp 1: Order of Convergence (CFL=0.8, T=15.0)")
axs[0].set_xlabel("dx")
axs[0].set_ylabel("Max Error")
axs[0].legend()

# Plot 2: Stability Identification
xs, u, u_ex, _, _ = run_lf(nx=100, CFL=0.98, T=15.0)
axs[1].plot(xs, u_ex.real, '--', label='Exact')
axs[1].plot(xs, u.real, label='Numerical')
axs[1].set_title("Exp 2: Stability (nx=100, CFL=0.98, T=15.0)")
axs[1].set_xlabel("x")
axs[1].legend()
xs, u, u_ex, _, _ = run_lf(nx=100, CFL=1.02, T=15.0)
axs[2].plot(xs, u_ex.real, '--', label='Exact')
axs[2].plot(xs, u.real, label='Numerical')
axs[2].set_title("Exp 2: Stability (nx=100, CFL=1.02, T=15.0)")
axs[2].set_xlabel("x")
axs[2].legend()

# Plot 3: CFL vs. Stability
axs[3].plot(cfl_test_vals, max_amplitudes, 'ro-')
axs[3].axvline(1.0, color='k', linestyle='--')
axs[3].set_yscale('log')
axs[3].set_title("Exp 3: CFL vs. Max Amplitudes (nx=100, T=15.0)")
axs[3].set_xlabel("CFL Number")
axs[3].set_ylabel("Max |u| (Log Scale)")

plt.tight_layout()
plt.show()

```



- As we can see in the graph, the Leapfrog scheme is order 2 convergence.
- For CFL , when we increase it from 0 to 1, the numerical wave approaches the exact wave. However, after some limit bigger than 1, the wave becomes oscillations. For nx , when we increase it, the numerical wave will approach the exact wave. But for CFL, c , increasing them will not affect the amplitudes. It will only increase the delay which represent as left-right shifting on the graph.
- From last graph, we can see that the norm blows up at some point around $CFL = 1.02$. After that, the scheme becomes significantly unstable.
- In general, the convergence order and stability conditions of Leapfrog scheme is almost the same as Lax-Wendroff. The only difference is that Leapfrog is more sensitive to CFL number.

3.5 FTCDS

```
[15]: # Experiment 1: Convergence (Error vs. dx)
nxs = [5, 10, 20, 40, 80]
dxs, errors = [], []
for n in nxs:
    _, _, _, dx, err = run_ftcds(n, CFL=0.4, T=10)
    dxs.append(dx)
    errors.append(err)

# Experiment 2 & 3: Stability and CFL Effects
cfl_test_vals = np.linspace(0.1, 1.2, 20)
max_amplitudes = []
for cfl in cfl_test_vals:
    _, u_final, _, _, _ = run_ftcds(nx=100, CFL=cfl, T=10)
    max_amplitudes.append(np.max(np.abs(u_final)))

# Visualization
fig, axs = plt.subplots(1, 4, figsize=(18, 5))

# Plot 1: Convergence
axs[0].loglog(dxs, errors, 'o-', label='ftcds Error')
```

```

axs[0].loglog(dxs, [d for d in dxs], '--', label='Order 1 (Slope=1)')
axs[0].set_title("Exp 1: Order of Convergence (CFL=0.8, T=15.0)")
axs[0].set_xlabel("dx")
axs[0].set_ylabel("Max Error")
axs[0].legend()

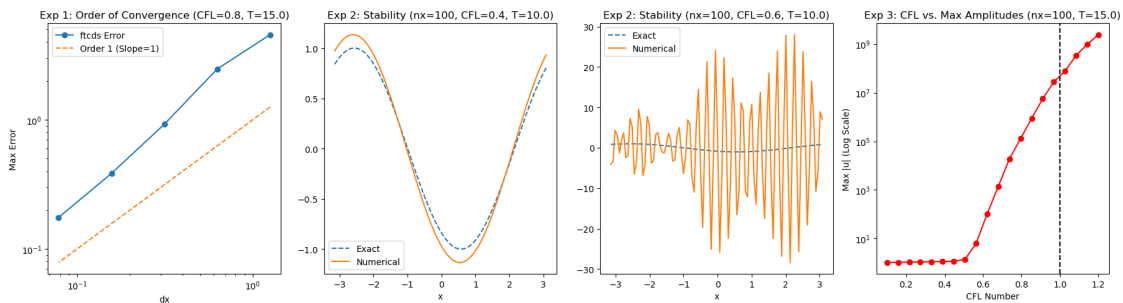
# Plot 2: Stability Identification
xs, u, u_ex, _, _ = run_ftcds(nx=100, CFL=0.4, T=10)
axs[1].plot(xs, u_ex.real, '--', label='Exact')
axs[1].plot(xs, u.real, label='Numerical')
axs[1].set_title("Exp 2: Stability (nx=100, CFL=0.4, T=10.0)")
axs[1].set_xlabel("x")
axs[1].legend()

xs, u, u_ex, _, _ = run_ftcds(nx=100, CFL=0.6, T=10)
axs[2].plot(xs, u_ex.real, '--', label='Exact')
axs[2].plot(xs, u.real, label='Numerical')
axs[2].set_title("Exp 2: Stability (nx=100, CFL=0.6, T=10.0)")
axs[2].set_xlabel("x")
axs[2].legend()

# Plot 3: CFL vs. Stability
axs[3].plot(cfl_test_vals, max_amplitudes, 'ro-')
axs[3].axvline(1.0, color='k', linestyle='--')
axs[3].set_yscale('log')
axs[3].set_title("Exp 3: CFL vs. Max Amplitudes (nx=100, T=15.0)")
axs[3].set_xlabel("CFL Number")
axs[3].set_ylabel("Max |u| (Log Scale)")

plt.tight_layout()
plt.show()

```



- By the graph, we get the order of convergence is 1 (However, as we discussed in lecture and I searched online, the order should be 2 for space).
- For CFL , when we increase it from 0 to 1, the numerical wave approaches the exact wave. However, after some limit bigger than 1, the wave becomes oscillations.

Similar for T , c and nx , when we increase them, there will be oscillations.

- From last graph, we can see that the norm blows up at some point around $CFL = 0.5$. After that, the scheme becomes significantly unstable.
- As I searched online, FTCDS scheme is unconditionally unstable. This may explain why the order shown by the graph is wrong and the CFL condition is unnormal (ie. blow up before $CFL = 1.0$).

3.6 Lax-Friedrichs

```
[16]: # Experiment 1: Convergence (Error vs. dx)
nxs = [50, 100, 200, 400, 800]
dxs, errors = [], []
for n in nxs:
    _, _, _, dx, err = run_laxf(n, CFL=0.8, T=15.0)
    dxs.append(dx)
    errors.append(err)

# Experiment 2 & 3: Stability and CFL Effects
cfl_test_vals = np.linspace(0.1, 1.2, 20)
max_amplitudes = []
for cfl in cfl_test_vals:
    _, u_final, _, _, _ = run_laxf(nx=100, CFL=cfl, T=15.0)
    max_amplitudes.append(np.max(np.abs(u_final)))

# Visualization
fig, axs = plt.subplots(1, 4, figsize=(18, 5))

# Plot 1: Convergence
axs[0].loglog(dxs, errors, 'o-', label='Lax-F Error')
axs[0].loglog(dxs, [d for d in dxs], '--', label='Order 1 (Slope=1)')
axs[0].set_title("Exp 1: Order of Convergence (CFL=0.8, T=15.0)")
axs[0].set_xlabel("dx")
axs[0].set_ylabel("Max Error")
axs[0].legend()

# Plot 2: Stability Identification
xs, u, u_ex, _, _ = run_laxf(nx=100, CFL=0.8, T=15.0)
axs[1].plot(xs, u_ex.real, '--', label='Exact')
axs[1].plot(xs, u.real, label='Numerical')
axs[1].set_title("Exp 2: Stability (nx=100, CFL=0.8, T=15.0)")
axs[1].set_xlabel("x")
axs[1].legend()

xs, u, u_ex, _, _ = run_laxf(nx=100, CFL=1.2, T=15.0)
axs[2].plot(xs, u_ex.real, '--', label='Exact')
axs[2].plot(xs, u.real, label='Numerical')
```

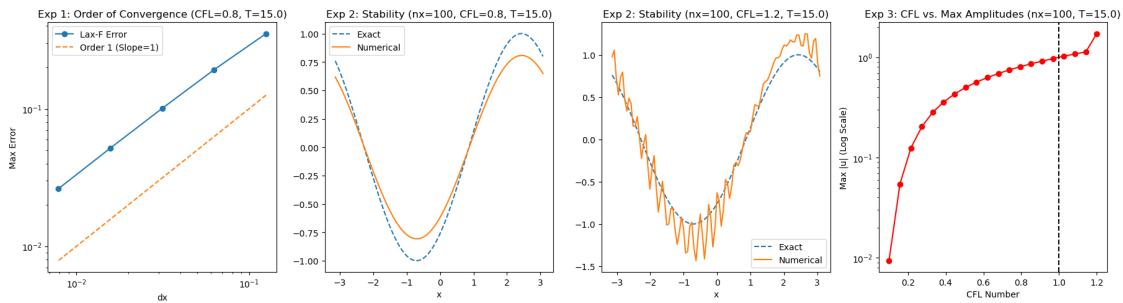
```

axs[2].set_title("Exp 2: Stability (nx=100, CFL=1.2, T=15.0)")
axs[2].set_xlabel("x")
axs[2].legend()

# Plot 3: CFL vs. Stability
axs[3].plot(cfl_test_vals, max_amplitudes, 'ro-')
axs[3].axvline(1.0, color='k', linestyle='--')
axs[3].set_yscale('log')
axs[3].set_title("Exp 3: CFL vs. Max Amplitudes (nx=100, T=15.0)")
axs[3].set_xlabel("CFL Number")
axs[3].set_ylabel("Max |u| (Log Scale)")

plt.tight_layout()
plt.show()

```



- As we can see in the graph, the Lax-Friedrichs scheme is order 1 convergence.
- For CFL , when we increase it from 0 to 1, the numerical wave approaches the exact wave. However, after some limit bigger than 1, the wave becomes oscillations. For T and c , when we increase them, the numerical wave will become flatter. For nx , when we increase it, the numerical wave will approach the exact wave.
- From last graph, we can see that the norm blows up at some point around $CFL = 1.15$. After that, the scheme becomes significantly unstable.
- In general, the performance of Lax-Friedrichs scheme is similar to the upwind scheme. But, it is a bit less accurate than upwind scheme.

3.7 RK2CD2

```

[17]: # Experiment 1: Convergence (Error vs. dx)
nxs = [5, 10, 20, 400, 80]
dxs, errors = [], []
for n in nxs:
    _, _, _, dx, err = run_rk2(n, CFL=0.8, T=5.0)
    dxs.append(dx)

```

```

errors.append(err)

# Experiment 2 & 3: Stability and CFL Effects
cfl_test_vals = np.linspace(0.1, 1.2, 20)
max_amplitudes = []
for cfl in cfl_test_vals:
    _, u_final, _, _, _ = run_rk2(nx=100, CFL=cfl, T=15.0)
    max_amplitudes.append(np.max(np.abs(u_final)))

# Visualization
fig, axs = plt.subplots(1, 4, figsize=(18, 5))

# Plot 1: Convergence
axs[0].loglog(dxs, errors, 'o-', label='RK2 Error')
axs[0].loglog(dxs, [d**2 for d in dxs], '--', label='Order 2 (Slope=2)')
axs[0].set_title("Exp 1: Order of Convergence (CFL=0.8, T=15.0)")
axs[0].set_xlabel("dx")
axs[0].set_ylabel("Max Error")
axs[0].legend()

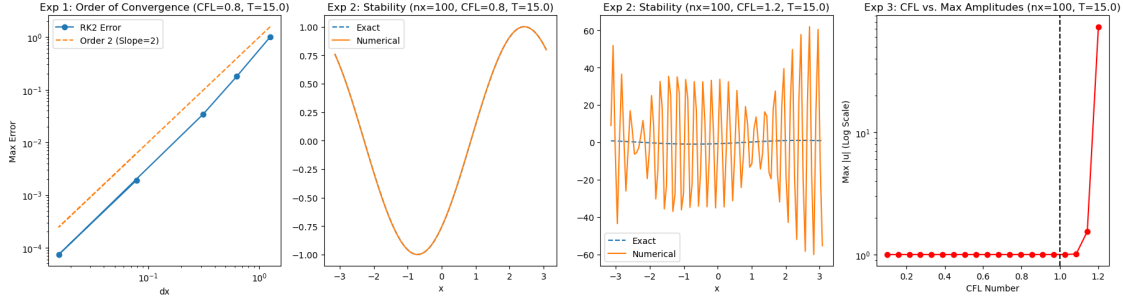
# Plot 2: Stability Identification
xs, u, u_ex, _, _ = run_rk2(nx=100, CFL=0.8, T=15.0, c=1)
axs[1].plot(xs, u_ex.real, '--', label='Exact')
axs[1].plot(xs, u.real, label='Numerical')
axs[1].set_title("Exp 2: Stability (nx=100, CFL=0.8, T=15.0)")
axs[1].set_xlabel("x")
axs[1].legend()

xs, u, u_ex, _, _ = run_rk2(nx=100, CFL=1.2, T=15.0)
axs[2].plot(xs, u_ex.real, '--', label='Exact')
axs[2].plot(xs, u.real, label='Numerical')
axs[2].set_title("Exp 2: Stability (nx=100, CFL=1.2, T=15.0)")
axs[2].set_xlabel("x")
axs[2].legend()

# Plot 3: CFL vs. Stability
axs[3].plot(cfl_test_vals, max_amplitudes, 'ro-')
axs[3].axvline(1.0, color='k', linestyle='--')
axs[3].set_yscale('log')
axs[3].set_title("Exp 3: CFL vs. Max Amplitudes (nx=100, T=15.0)")
axs[3].set_xlabel("CFL Number")
axs[3].set_ylabel("Max |u| (Log Scale)")

plt.tight_layout()
plt.show()

```

- As we can see in the graph, the RK4CD4 scheme is order 2 convergence.
- For CFL , when we increase it from 0 to 1, the numerical wave approaches the exact wave. However, after some limit bigger than 1, the wave becomes oscillations. Similar for T, c and nx , when we increase them, it creates oscillations.
- From last graph, we can see that the norm blows up at some point around $CFL = 1.15$. After that, the scheme becomes significantly unstable.
- In general, RK2CD2 is very accurate when it's stable, but the stable condition is more strict (ie. CFL, T, c, nx has to be small since increasing any of them will cause unstable).

3.8 RK4CD4

```
[18]: # Experiment 1: Convergence (Error vs. dx)
nxs = [5, 10, 20, 40, 80]
dxs, errors = [], []
for n in nxs:
    _, _, _, dx, err = run_rk4(n, CFL=0.8, T=0.005)
    dxs.append(dx)
    errors.append(err)

# Experiment 2 & 3: Stability and CFL Effects
cfl_test_vals = np.linspace(0.5, 2.5, 20)
max_amplitudes = []
for cfl in cfl_test_vals:
    _, u_final, _, _, _ = run_rk4(nx=100, CFL=cfl, T=15.0)
    max_amplitudes.append(np.max(np.abs(u_final)))

# Visualization
fig, axs = plt.subplots(1, 4, figsize=(18, 5))

# Plot 1: Convergence
axs[0].loglog(dxs, errors, 'o-', label='RK4 Error')
axs[0].loglog(dxs, [d**4 for d in dxs], '--', label='Order 4 (Slope=4)')
axs[0].set_title("Exp 1: Order of Convergence (CFL=0.8, T=15.0)")
```

```

axs[0].set_xlabel("dx")
axs[0].set_ylabel("Max Error")
axs[0].legend()

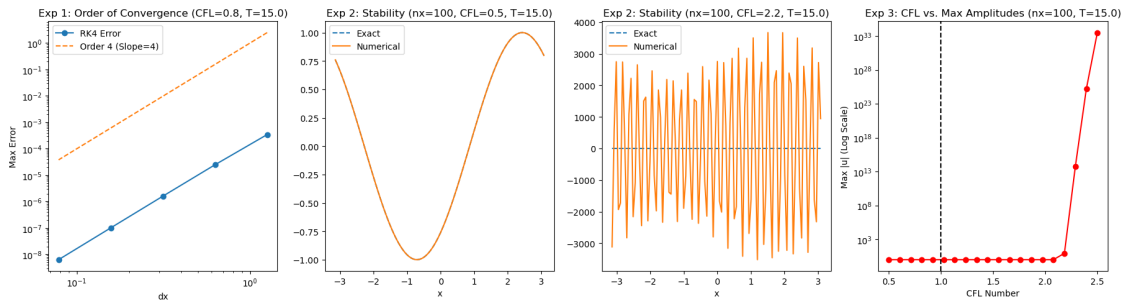
# Plot 2: Stability Identification
xs, u, u_ex, _, _ = run_rk4(nx=100, CFL=0.8, T=15)
axs[1].plot(xs, u_ex.real, '--', label='Exact')
axs[1].plot(xs, u.real, label='Numerical')
axs[1].set_title("Exp 2: Stability (nx=100, CFL=0.5, T=15.0)")
axs[1].set_xlabel("x")
axs[1].legend()

xs, u, u_ex, _, _ = run_rk4(nx=100, CFL=2.2, T=15)
axs[2].plot(xs, u_ex.real, '--', label='Exact')
axs[2].plot(xs, u.real, label='Numerical')
axs[2].set_title("Exp 2: Stability (nx=100, CFL=2.2, T=15.0)")
axs[2].set_xlabel("x")
axs[2].legend()

# Plot 3: CFL vs. Stability
axs[3].plot(cfl_test_vals, max_amplitudes, 'ro-')
axs[3].axvline(1.0, color='k', linestyle='--')
axs[3].set_yscale('log')
axs[3].set_title("Exp 3: CFL vs. Max Amplitudes (nx=100, T=15.0)")
axs[3].set_xlabel("CFL Number")
axs[3].set_ylabel("Max |u| (Log Scale)")

plt.tight_layout()
plt.show()

```



- As we can see in the graph, the RK4CD4 scheme is order 4 convergence.
- For CFL , when we increase it from 0 to 1, the numerical wave approaches the exact wave. However, after some limit bigger than 1, the wave becomes oscillations, which means the error blows up.
However, for T, c and nx , when we increase them, the numerical values are still accurate, and the error increases slowly.

- From last graph, we can see that the norm blows up at some point around $CFL = 2$. After that, the scheme becomes significantly unstable.
- In general, RK4CD4 is very accurate when it's stable. The stability won't be affected by values of T, c, nx that much, while CFL still plays an important role.